

LabReSiD26 - Hands-On 2

Davide Ferrara - 518629

1 Esercizio 1: Prova ad immaginare il funzionamento su rete reale

1.1 Dove si trova esattamente il problema della frammentazione nel client e nel server?

Il problema non è dato dal codice in sé ma dal fatto che tra Client e Server non è stabilito alcun protocollo di comunicazione. TCP è un protocollo che garantisce l'invio ordinato di un flusso di byte (stream-oriented), non esiste il concetto di messaggio in TCP.

Dal manuale di TCP (`man 7 tcp`):

TCP guarantees that the data arrives in order and retransmits lost packets. It generates and checks a per-packet checksum to catch transmission errors. TCP does not preserve record boundaries.

1.2 Perché avviene in trasmissione?

TCP può spezzare i dati in più segmenti per via del MSS (Maximum Segment Size), negoziato durante il 3-way handshake. Inoltre il Nagle's algorithm può raggruppare più `send()` in un unico segmento per ridurre l'overhead di rete.

1.3 Perché avviene in ricezione?

`recv()` restituisce quanti byte sono disponibili nel receive buffer del kernel in quel momento, senza aspettare un messaggio completo. Possono arrivare meno byte del previsto (segmento frammentato) o più byte del previsto (due `send()` fusi in un unico segmento).

2 Esercizio 2: Scrivi recv_all()

```
1 ssize_t recv_all(int sockfd, void *buf, size_t n) {
2     ssize_t r = 0;
3
4     /* cicla finche' non ha letto esattamente n byte */
5     while ((size_t)r < n) {
6         /* aritmetica di puntatori: (char*)buf + r sposta il
7          cursore
8          sui byte gia' ricevuti; n-r e' quanto manca
9          ancora */
10        ssize_t received = recv(sockfd, (char *)buf + r, n -
11                                r, 0);
12        if (received == -1) { /* errore di sistema */
13            perror("[ERROR] recv errore: ");
14            return -1;
15        }
16        if (received == 0) { /* peer ha chiuso la
17                             connessione (FIN) */
18            printf("[DEBUG] connessione chiusa!\n");
19            return 0;
20        }
21        r += received; /* aggiorna il cursore */
22    }
23
24    return n; /* restituisce n byte letti con successo */
25 }
```

Listing 1: recv_all() – implementazione

```
1 ssize_t send_all(int sockfd, void *buf, size_t n) {
2     ssize_t s = 0;
3
4     /* cicla finche' non ha inviato esattamente n byte */
5     while ((size_t)s < n) {
6         /* aritmetica di puntatori: (char*)buf + s sposta il
7          cursore
8          sui byte gia' inviati; n-s e' quanto manca ancora
9          */
10        ssize_t sent = send(sockfd, (char *)buf + s, n - s,
11                             0);
12        if (sent == -1) { /* errore di sistema */
13            perror("[ERROR] send errore: ");
14            return -1;
15        }
16        s += sent;
17    }
18
19    return n;
20 }
```

```

12     }
13     if (sent == 0) { /* connessione chiusa */
14         printf("[DEBUG] connessione chiusa!\n");
15         return 0;
16     }
17     s += sent; /* aggiorna il cursore */
18 }
19
20 return n; /* restituisce n byte inviati con successo
21 */

```

Listing 2: send_all() – implementazione

L'implementazione di questa funzione non risolve il problema ugualmente perché i messaggi sono di lunghezza variabile, serve inviare anche questa informazione al server dal client.

3 Esercizio 3: Il problema della lunghezza

3.1 a) Il mittente deve mandare prima la lunghezza o prima il payload?

Il mittente deve mandare prima la lunghezza, così il ricevitore legge i 4 byte di header, conosce quanti byte aspettarsi e chiama `recv_all()` con quella lunghezza esatta.

3.2 b) Dimensione massima di un singolo messaggio con `uint32_t`

`uint32_t` va da 0 a $2^{32} - 1$, quindi la dimensione massima del payload è $2^{32} - 1$ byte, circa 4 GB.

3.3 c) Perché è necessario usare `htonl()` sul campo lunghezza?

Macchine diverse potrebbero usare endianness diverse (little-endian su x86, big-endian su architetture a 64 bit). Il network byte order è big-endian per convenzione. Senza `htonl()`, un client little-endian invia i byte in ordine inverso e il server interpreta un numero completamente sbagliato.

4 Esercizio 4: Implementa il protocollo

La struct `message` usata da client e server:

```
1 typedef struct message {  
2     uint32_t len;  
3     char *payload;  
4 } message;
```

Listing 3: Struttura message

Il mittente invia prima i 4 byte di lunghezza in network byte order (`htonl`), poi il payload. Il ricevitore legge i 4 byte, converte con `ntohl`, alloca esattamente `len+1` byte e chiama `recv_all()` con quella lunghezza.

```
1 int send_message(int sockfd, const message *msg) {  
2     uint32_t net_len = htonl(msg->len);  
3     if (send_all(sockfd, &net_len, sizeof(uint32_t)) < 0)  
4         return -1;  
5     if (send_all(sockfd, (void *)msg->payload, msg->len) <  
6         0)  
7         return -1;  
8     return 0;  
9 }
```

Listing 4: `send_message()` – client

Il ricevente legge i primi 4 byte, la lunghezza, e li converte in host long, alloca la memoria (`len + 1`) e termina la stringa.

```
1 int recv_message(int sockfd, message *msg) {  
2     if (recv_all(sockfd, &msg->len, sizeof(uint32_t)) <=  
3         0)  
4         return -1;  
5     msg->len = ntohl(msg->len);  
6     msg->payload = (char *)malloc(sizeof(char) * msg->len  
7         + 1);  
8     if (recv_all(sockfd, msg->payload, msg->len) <= 0)  
9         return -1;  
10    msg->payload[msg->len] = '\0';  
11    return 0;  
12 }
```

Listing 5: `recv_message()` – server

Output Con un messaggio piccolo "Ciao col protocollo!" (20 byte):

```

1 [DEBUG] Lunghezza divina_commedia.txt: 64000 bytes
2 [CLIENT.c] Inviando: Ciao col protocollo!
3 [DEBUG] r=4
4 [DEBUG] r=20

```

Listing 6: Output – messaggio piccolo

I [DEBUG] r=4 e r=20 confermano che `send_all()` trasmette prima i 4 byte di header, poi i 20 byte di payload.

Test con messaggio da 64 KB Il client invia un testo della *Divina Commedia* (64000 byte) in un singolo messaggio. Il server riceve correttamente l'intero payload grazie a `recv_all()` che cicla fino a leggere esattamente `len` byte:

```

1 Rispuosemi: "Non omo, omo gia' fui,
2 e li parenti miei furon lombardi,
3 mantoani per patria ambedui.
4 [...]
5 [DEBUG] connessione chiusa!

```

Listing 7: Output server – payload 64 KB

5 Esercizio 5: Aggiungi il checksum

Il checksum XOR viene calcolato su tutti i byte del payload prima dell'invio e verificato dal server dopo la ricezione. Il formato del messaggio diventa:

| len (4 byte) | payload (len byte) | checksum (1 byte) |

La struct `message` viene estesa con il campo `checksum`:

```

1 typedef struct message {
2     uint32_t len;
3     char *payload;
4     uint8_t checksum;
5 } message;

```

Listing 8: Struct message con checksum

Il checksum viene calcolato come lo XOR di tutti i caratteri del payload.

```
1 uint8_t checksum(char *payload) {
2     uint8_t checksum = 0;
3     for (size_t i = 0; i < strlen(payload); i++) {
4         checksum ^= (uint8_t)payload[i];
5     }
6     return checksum;
7 }
```

Listing 9: Calcolo checksum XOR

Il client calcola il checksum in `create_message()` e lo invia come ultimo byte dopo il payload in `send_message()`:

```
1 /* In create_message(): */
2 msg->checksum = checksum(msg->payload);
3
4 /* In send_message(): */
5 printf("[CLIENT.c] Checksum inviato: %d\n", msg->
6     checksum);
7 if (send_all(sockfd, (void *)&msg->checksum, sizeof(
8     uint8_t)) < 0)
9     return -1;
```

Listing 10: Invio checksum – client

Il server riceve il byte di checksum, calcola il proprio e confronta:

```
1 if (recv_all(sockfd, &msg->checksum, sizeof(uint8_t)) <=
2     0)
3     return -1;
4
5 uint8_t computed = checksum(msg->payload);
6 printf("[SERVER.c] Checksum ricevuto: %d | Checksum
7     calcolato: %d\n",
8     msg->checksum, computed);
9
10 if (!(checksum(msg->payload) && msg->checksum)) {
11     printf("[DEBUG] Checksum fallito!\n");
12     return -1;
13 }
14 printf("[DEBUG] Checksum corretto, il messaggio e'
15     integro!\n");
```

Listing 11: Verifica checksum – server

Il server tronca il payload stampato a 64 caratteri per leggibilità (stampare 64KB di divina commedia nel terminal sarebbe impossibile), il checksum controlla comunque la validità del messaggio.

```
1 if (strlen(msg->payload) > 64) {  
2     msg->payload[61] = '.';  
3     msg->payload[62] = '.';  
4     msg->payload[63] = '.';  
5     msg->payload[64] = '\\0';  
6 }
```

Listing 12: Troncamento output – server

Output con file da 64000 byte (*Divina Commedia*):

```
1 [DEBUG] r=4  
2 [DEBUG] r=64000  
3 [DEBUG.c] len: 64000  
4 [DEBUG] r=1  
5 [SERVER.c] Checksum ricevuto: 54 | Checksum calcolato:  
6     54  
7 [DEBUG] Checksum corretto, il messaggio e' integro!  
8 [SERVER.c] Messaggio: Nel mezzo del cammin di nostra  
vita  
mi ritrovai per una selva...
```

Listing 13: Output – checksum su 64 KB

r=4 riceve il campo lunghezza, r=64000 l'intero payload, r=1 il byte di checksum. I valori 54 coincidono: il messaggio è arrivato integro.

6 Domande di riflessione

Q1. Perché il codice funzionava in locale ma può fallire su rete reale?

In loopback il kernel consegna i dati in un unico chunk mentre nella rete reale TCP può spezzarli.

Q2. Differenza tra `recv()` che ritorna 0 e `recv()` che ritorna -1

`recv()` ritorna 0 se il client ha effettuato uno shutdown in ordine (FIN ricevuto). `recv()` ritorna -1 nel caso di errori come una chiusura improvvisa della connessione.

Q3. Dimensione massima del payload con uint16_t nel campo lunghezza

uint16_t corrisponde a un intero senza segno da 16 bit quindi $\max = 2^{16} - 1 = 65535$ byte di payload.

Q4. Il protocollo è adatto a trasferire file binari?

Il protocollo in generale sì, ma non nel mio codice perché nel checksum uso `strlen()`, che si ferma al primo byte `0x00`. I file binari contengono null byte essendo una sequenza di 0 e 1, quindi il checksum sarebbe errato. Anche la struttura **message** non sarebbe adatta per via del buffer di tipo `char*`.